```
// g++ -O2 -std=c++17 lemma23_verify_exact.cpp -lgmp -o lemma23_verify
// ./lemma23_verify > lemma23_verification_output.txt
// Exact finite certificate for Lemma 2.3, equation (2.26).
// d1(k)=p(k)-p(k-1), d2(k)=p(k)-p(k-2), and p(k)=0 for k<0.
// D(t,0): 72<=t<10000. R0: 3<=s<=34. Rs: 35<=s<10000.
// All mathematical calculations use exact GMP integers.
// Positive denominators are checked before rational comparisons are cleared.

#include <gmp.h>
#include <algorithm>
#include <chrono>
#include <iostream>
#include <vector>

struct Integer
{
    mpz_t z;

    Integer()
    {
        mpz_init(z);
    }

    ~Integer()
    {
        mpz_clear(z);
    }

    Integer(const Integer &) = delete;
    Integer &operator=(const Integer &) = delete;
};

int checks = 0, failures = 0;

void check(bool condition, const char *description)
{
    ++checks;
    if (!condition)
        ++failures;
    std::cout << (condition ? "PASS " : "FAIL ") << description << '\n';
}

// Store an exact minimum and its location, including the number of ties.
struct Minimum
{
    Integer value;
    int index = -1, ties = 0;

    void update(const mpz_t candidate, int current)
    {
        if (index < 0 || mpz_cmp(candidate, value.z) < 0)
        {
            mpz_set(value.z, candidate);
            index = current;
            ties = 1;
        }
        else if (mpz_cmp(candidate, value.z) == 0)
            ++ties;
    }

    void print(const char *label) const
    {
        gmp_printf("MINIMUM %s=%Zd index=%d ties=%d\n",
                   label, value.z, index, ties);
    }
};

// For R=A/B, B>0, calculate the two integer margins:
//   margin1 = B*d2(s)-A*d2(s-2),
//   margin2 = B^2*q_s(A/B).
```

```
void ratio_margins(mpz_t margin1, mpz_t margin2, const mpz_t A,
                   const mpz_t B, int s, const std::vector<Integer> &d2)
{
    Integer temp;
    mpz_mul(margin1, B, d2[s].z);
    mpz_submul(margin1, A, d2[s - 2].z);
    mpz_mul(margin2, B, B);
    mpz_mul(margin2, margin2, d2[s + 2].z);
    mpz_mul(temp.z, A, B);
    mpz_mul(temp.z, temp.z, d2[s].z);
    mpz_mul_ui(temp.z, temp.z, 2);
    mpz_sub(margin2, margin2, temp.z);
    mpz_mul(temp.z, A, A);
    mpz_addmul(margin2, temp.z, d2[s - 2].z);
}

int verify_ratios(const char *name, int first, int last,
                  const std::vector<Integer> &d1,
                  const std::vector<Integer> &d2, bool fixed_ratio)
{
    int tested = 0, invalid = 0, failed1 = 0, failed2 = 0;
    Minimum min1, min2;
    Integer margin1, margin2;
    for (int lo = first; lo <= last; lo += 1000)
    {
        const int hi = std::min(last, lo + 999);
        int bad = 0;
        for (int s = lo; s <= hi; ++s)
        {
            const int a_index = fixed_ratio ? 72 : 2 * s + 4;
            const int b_index = fixed_ratio ? 71 : 2 * s + 3;
            const auto &A = d1[a_index];
            const auto &B = d1[b_index];
            if (mpz_sgn(B.z) <= 0 || mpz_cmp(A.z, B.z) < 0)
            {
                ++invalid;
                ++bad;
            }
            ratio_margins(margin1.z, margin2.z, A.z, B.z, s, d2);
            if (mpz_sgn(margin1.z) <= 0)
            {
                ++failed1;
                ++bad;
            }
            if (mpz_sgn(margin2.z) <= 0)
            {
                ++failed2;
                ++bad;
            }
            min1.update(margin1.z, s);
            min2.update(margin2.z, s);
            ++tested;
        }
        std::cout << (bad ? "BLOCK_FAIL " : "BLOCK_PASS ") << name
                  << " s=" << lo << ".." << hi
                  << " cases=" << hi - lo + 1
                  << " inequalities=" << 2 * (hi - lo + 1)
                  << " failures=" << bad << '\n';
    }
    const bool ok = tested == last - first + 1 && invalid == 0 &&
                    failed1 == 0 && failed2 == 0;
    check(ok, fixed_ratio ? "R0: positive denominators, R>=1, both strict inequalities"
                          : "Rs: positive denominators, R>=1, both strict inequalities");
    min1.print(fixed_ratio ? "R0_cleared_margin1" : "Rs_cleared_margin1");
    min2.print(fixed_ratio ? "R0_cleared_margin2" : "Rs_cleared_margin2");
    return tested;
}

int main()
{
```

```
const auto start = std::chrono::steady_clock::now();
constexpr int limit = 20002;
std::vector<Integer> p(limit + 1), product(limit + 1);
std::vector<Integer> d1(limit + 1), d2(limit + 1);
mpz_set_ui(p[0].z, 1);
for (int n = 1; n <= limit; ++n)
{
    for (int r = 1;; ++r)
    {
        const int a = r * (3 * r - 1) / 2;
        if (a > n)
            break;
        const int b = a + r;
        if (r & 1)
        {
            mpz_add(p[n].z, p[n].z, p[n - a].z);
            if (b <= n)
                mpz_add(p[n].z, p[n].z, p[n - b].z);
        }
        else
        {
            mpz_sub(p[n].z, p[n].z, p[n - a].z);
            if (b <= n)
                mpz_sub(p[n].z, p[n].z, p[n - b].z);
        }
    }
}
std::cout << "PARTITIONS_READY first=0 last=20002 count=20003\n";

// Independently check the entire table by the partition product.
mpz_set_ui(product[0].z, 1);
for (int part = 1; part <= limit; ++part)
    for (int n = part; n <= limit; ++n)
        mpz_add(product[n].z, product[n].z, product[n - part].z);
int mismatches = 0;
for (int n = 0; n <= limit; ++n)
{
    if (mpz_cmp(p[n].z, product[n].z) != 0)
        ++mismatches;
    mpz_set(d1[n].z, p[n].z);
    mpz_set(d2[n].z, p[n].z);
    if (n >= 1)
        mpz_sub(d1[n].z, d1[n].z, p[n - 1].z);
    if (n >= 2)
        mpz_sub(d2[n].z, d2[n].z, p[n - 2].z);
}
check(mismatches == 0, "independent_partition_product n=0..20002 count=20003");
if (mismatches)
    return 1;

Integer expected, determinant, direct, a, b;
mpz_mul_ui(expected.z, d1[72].z, 609237);
mpz_mul_ui(direct.z, d1[71].z, 695578);
check(mpz_cmp(expected.z, direct.z) == 0,
      "R0=d1(72)/d1(71)=695578/609237");
gmp_printf("VALUE R0=%Zd/%Zd\n", d1[72].z, d1[71].z);
check(mpz_cmp_ui(d2[0].z, 1) == 0 && mpz_cmp_ui(d2[2].z, 1) == 0 &&
          mpz_cmp_ui(d2[4].z, 3) == 0,
      "q_2(R)=(R-1)^2+2>0 for every real R");

int compared = 0, nonpositive = 0, identity_failures = 0;
Minimum minimum_D;
for (int lo = 72; lo <= 9999; lo += 1000)
{
    const int hi = std::min(9999, lo + 999);
    int bad = 0;
    for (int t = lo; t <= hi; ++t)
    {
        // Direct substitution in D(t,0), using d2(0)=d2(2)=1.
        mpz_sub_ui(a.z, d1[t].z, 1);
```

```
            mpz_sub_ui(b.z, d1[t + 1].z, 1);
            mpz_mul(direct.z, a.z, a.z);
            mpz_submul(direct.z, b.z, d1[t - 1].z);
            // Independently cross-check the displayed expanded identity.
            mpz_mul(determinant.z, d1[t].z, d1[t].z);
            mpz_submul(determinant.z, d1[t - 1].z, d1[t + 1].z);
            mpz_submul_ui(determinant.z, d1[t].z, 2);
            mpz_add(determinant.z, determinant.z, d1[t - 1].z);
            mpz_add_ui(determinant.z, determinant.z, 1);
            if (mpz_cmp(direct.z, determinant.z) != 0)
                ++identity_failures;
            if (mpz_sgn(direct.z) <= 0)
            {
                ++nonpositive;
                ++bad;
            }
            minimum_D.update(direct.z, t);
            ++compared;
        }
        std::cout << (bad ? "BLOCK_FAIL " : "BLOCK_PASS ")
                  << "D(t,0) t=" << lo << ".." << hi
                  << " inequalities=" << hi - lo + 1 << " failures=" << bad << '\n';
    }
    check(compared == 9928 && nonpositive == 0,
          "D(t,0)>0 for all 72<=t<10000; cases=9928");
    check(identity_failures == 0, "direct and expanded D(t,0) agree at all 9928 indices");
    minimum_D.print("D(t,0)");

    const int fixed_cases = verify_ratios("R0", 3, 34, d1, d2, true);
    const int varying_cases = verify_ratios("Rs", 35, 9999, d1, d2, false);
    check(fixed_cases == 32 && varying_cases == 9965,
          "ratio ranges cover every integer s=3..9999 exactly once");
    const int inequalities = compared + 2 * (fixed_cases + varying_cases);
    check(inequalities == 29922, "core inequality count = 29922");
    const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
                             std::chrono::steady_clock::now() - start).count();
    std::cout << (failures ? "COMPLETE_FAIL" : "COMPLETE_PASS")
              << " core_inequalities=" << inequalities
              << " identity_checks=" << compared
              << " group_checks=" << checks << " failed=" << failures
              << " elapsed_us=" << elapsed << '\n';
    return failures ? 1 : 0;
}
```